MALWARE ANALYSIS & INCIDENT FORENSICS

Appunti Integrati di Lezione – Lezione 2: Basic Static Analysis

1. OBIETTIVI DELL'ANALISI DEL MALWARE

La comprensione approfondita del funzionamento del malware è fondamentale per due scopi principali in ambito aziendale ed istituzionale:

- **Monitoraggio e Prevenzione degli Attacchi**: Identificare pattern di attacco ricorrenti per costruire regole di detection automatiche (SIEM/IDS) e progettare sistemi IT più resilienti.
- **Incident Response (Risposta agli Incidenti)**: Comprendere esattamente l'accaduto, identificare tutti gli asset compromessi (macchine, file, credenziali), contenere la diffusione, eradicare l'attacco e ripristinare la funzionalità (Recovery).

```
NIST Incident Response Cycle:
[ Preparation ] ---> [ Detection & Analysis ] <---> [ Containment, Eradication & Recovery ]
---> [ Post-Incident Activity ]
```

Tipologie di Firme (Signatures)

- **Host-based Signatures**: Identificano tracce sul singolo host (nomi di file creati, processi in esecuzione, chiavi di Registro Windows modificate). Facili da alterare per l'attaccante.
- **Network Signatures**: Basate sull'analisi del traffico di rete (richieste a domini C2, User-Agent particolari, pattern di comunicazione).
- **Behavioral Signatures (Firme Comportamentali)**: Descrivono la sequenza di azioni compiute dal malware (es. scrive un file in `/temp`, contatta un social network, scarica comandi). **Sono le più efficaci**: richiedono un costo/sforzo elevatissimo per essere modificate dall'attaccante rispetto alle firme statiche.

> ⚠️ **REGOLE DI SICUREZZA PER L'ANALISTA (WARNING):**
>
> 1. Effettuare **sempre backup offline** disconnessi dei propri dati importanti.
> 2. Analizzare il malware **esclusivamente in Macchine Virtuali isolate** dal sistema host (con rete disattivata se non strettamente necessaria).
> 3. **MAI contattare direttamente gli indirizzi IP/Domini esterni trovati nei sample reali**: dietro c'è un criminale e significherebbe allertarlo della scoperta (equivalente elettronico di bussare a casa del ladro).

2. CATEGORIZZAZIONE DELLE TECNICHE DI ANALISI

Livello / Tipo	Analisi Statica (Static)	Analisi Dinamica (Dynamic)
Basic (Base)	Esame delle caratteristiche del file senza eseguirlo (stringhe, hash, intestazioni PE). Veloce ma aggirabile da malware avanzato.	Esecuzione del malware in ambiente controllato/VM e monitoraggio delle interazioni con il SO (Process Monitor, RegShot).
Advanced (Avanzata)	Reverse Engineering del codice binario mediante disassemblatore (IDA Pro, Ghidra). Complessa, richiede x86 assembly.	Esecuzione controllata istruzione per istruzione in un Debugger per ispezionare memoria, registri e Stack.

Trade-Off e Complementarità tra Statica e Dinamica

- **Limitazioni della sola Analisi Statica**: Il codice è spesso offuscato o cifrato. Senza l'ambiente di runtime non è possibile conoscere lo stato effettivo della memoria e dello stack.
- **Limitazioni della sola Analisi Dinamica**: Si osserva **un solo percorso di esecuzione (execution path)**. Se il malware contiene condizioni particolari (es. controlli di data, presenza di determinati file o tecniche di evasione), i rami non attivati rimarranno invisibili.
- **Regola Pratica**: Si usano sempre prima gli strumenti **Basic** per avere una panoramica (cheap & fast), riservando gli strumenti **Advanced** (expensive in termini di tempo) solo alle porzioni di codice chiave.

3. TECNICHE DI BASIC STATIC ANALYSIS

A. Scansione Antivirus e VirusTotal

- **Primo passo ovvio**: Utile per verificare se il file è già noto.
- **Rischio Operativo di VirusTotal**: Se si carica un sample riservato/mirato su VirusTotal, l'attaccante che monitora l'hash tramite le API o la funzione di ricerca scoprirà che l'infezione è stata rilevata e cambierà strategia di attacco.

B. Hashing dei Sample

- L'hash (MD5, SHA-1, SHA-256) riduce un file di qualsiasi dimensione ad una firma a lunghezza fissa che lo identifica in modo univoco.
- **Differenza tra Sample e Malware**: La codebase di un malware (il suo comportamento) è unica. Se l'attaccante modifica una sola riga irrilevante di codice e ricompila, ottiene un **sample diverso** con un hash completamente nuovo, aggirando le firme tradizionali.
- **Imphash (Import Hash)**: Hash calcolato unicamente sulla tabella delle funzioni e librerie importate. Molto efficace per raggruppare sample appartenenti alla stessa famiglia di malware.

C. Estrarre Stringhe (Strings Command)

Una stringa è una sequenza di caratteri stampabili terminata da un valore null (`0x00`). Può essere in formato **ASCII** (8-bit per carattere) o **Unicode/Wide Character** (16-bit per carattere, dove i caratteri ASCII sono seguiti da `0x00`).

- Lo strumento `strings` scansiona il file alla ricerca di sequenze di caratteri (default ≥ 3 o 4 caratteri).
- Fornisce indicazioni cruciali: indirizzi IP, URL, nomi di file, chiavi di registro, messaggi di errore e nomi di funzioni API usate dal malware.

```
ASCII Representation:   [ 'B' (0x42) ] [ 'A' (0x41) ] [ 'D' (0x44) ] [ NULL (0x00) ]
Unicode Representation: [ 'B' (0x42) ] [ 0x00 ] [ 'A' (0x41) ] [ 0x00 ] [ 'D' (0x44) ] [ 0x0
0 ] [ NULL (0x00) ] [ 0x00 ]
```

D. Offuscamento e Packing

I creatori di malware comprimono/cifrano l'eseguibile per nascondere il codice e le stringhe agli strumenti di analisi.

- **Funzionamento di un Packer**: L'eseguibile originale viene compresso ed incapsulato in una nuova struttura a cui viene aggiunto un piccolo codice chiamato **Wrapper/Unpacker Routine**. All'avvio, il wrapper decomprime in memoria il codice originale e vi trasferisce l'esecuzione (Original Entry Point - OEP).
- **Rilevamento dei Packer**: Si utilizzano strumenti come **PEiD** o **Detect It Easy (DIE)** che confrontano le firme delle sezioni dell'eseguibile (es. sezioni chiamate `UPX0` , `UPX1`).

```
Eseguibile Originale (Stringhe e Codice visibili)  --->  [ Wrapper / Stub ] (Decomprime in RAM)
                                                                [ Executable Compresso / Offuscato ]
```

4. IL FORMATO PE (PORTABLE EXECUTABLE)

Il formato **PE (Portable Executable)** è la struttura standard usata dal Loader di Windows per caricare ed eseguire file binari (`.exe`), librerie dinamiche (`.dll`) e driver (`.sys`).

Struttura Generica di un File PE

```
+-----------------------------------------------------+
| DOS MZ Header (Inizia con 'MZ' / 0x5A4D)            |
+-----------------------------------------------------+
| DOS Stub ("This program must be run under Win32")   |
+-----------------------------------------------------+
| PE Header (Inizia con 'PE\0\0' / 0x50450000)        |
|   ├── COFF File Header                              |
|   └── Optional Header                               |
+-----------------------------------------------------+
| Section Table (Elenco e descrittori delle sezioni)  |
+-----------------------------------------------------+
| Section 1 (.text / CODE) -> Codice Eseguibile       |
+-----------------------------------------------------+
| Section 2 (.data / .rdata) -> Dati / Variabili Glob.|
+-----------------------------------------------------+
| Section N (.rsrc, .idata, .edata) -> Risorse/Import |
+-----------------------------------------------------+
```

Analisi dei Componenti Principali del PE Header

1. DOS Header & DOS Stub

- I primi 64 byte costituiscono il **DOS Header**. I primi 2 byte contengono la firma "magic" `MZ` (`0x5A4D`).
- L'ultimo campo del DOS Header (all'offset `0x3C` , chiamato `e_lfanew`) contiene un puntatore all'offset di inizio del vero **PE Header**.
- Il **DOS Stub** è un piccolo programma legacy eseguito se il file viene avviato in MS-DOS, stampando semplicemente il messaggio *"This program must be run under Microsoft Windows"*.

2. PE Header (COFF & Optional Header)

- Inizia con la firma a 4 byte `PE\0\0` (`0x50450000`).
- **COFF File Header**: Contiene metadati cruciali tra cui `NumberOfSections` (numero di sezioni presenti nel file) e `Characteristics` (flag che indicano se il file è un `.exe` o una `.dll`).
- **Optional Header** (Presente in quasi tutti i file eseguibili):
  - `AddressOfEntryPoint (RVA)` : Indirizzo relativo (RVA) della prima istruzione da eseguire.
  - `ImageBase` : Indirizzo di memoria preferito in cui caricare il file (solitamente `0x00400000` per i file `.exe` a 32-bit).
  - `SectionAlignment` : Allineamento delle sezioni quando sono caricate in memoria RAM (tipicamente $4\text{ KB} = 0x1000$).
  - `FileAlignment` : Allineamento dei dati delle sezioni nel file memorizzato su disco (tipicamente $512\text{ byte} = 0x200$).

- ◦ `SizeOfImage`: Dimensione totale occupying in memoria RAM dal programma caricato.

3. Section Table & Sezioni Comuni

La **Section Table** contiene una serie di descrittori (40 byte ciascuno) che specificano le caratteristiche di ogni sezione:

- `Name`: Nome della sezione (es. `.text`, `.data`). Nota: i nomi sono pura convenzione e possono essere rinominati dal malware!
- `VirtualSize`: Dimensione occupata dalla sezione quando viene caricata in memoria RAM.
- `SizeOfRawData`: Dimensione occupata dalla sezione su disco.
- `Characteristics`: Permessi della sezione (Leggibile, Scrivibile, Eseguibile).

> 💡 **INDICATORE DI PACKING NELLE SEZIONI:** Nei programmi normali, `VirtualSize` e `SizeOfRawData` per la sezione `.text` sono quasi uguali. Se in una sezione si osserva una **VirtualSize molto più grande della SizeOfRawData** (o `SizeOfRawData = 0`), l'eseguibile è quasi certamente **PACKED** (il wrapper allocherà spazio in RAM per decomprimere il codice che su disco non occupa spazio).

Nomi delle Sezioni Comuni

Sezione	Contenuto e Descrizione
`.text` / `CODE`	Contiene il codice eseguibile del programma. Di norma è contrassegnata come Eseguibile/Leggibile e NON Scrivibile.
`.data` / `.bss`	Contiene variabili globali e dati inizializzati (`.data`) o non inizializzati (`.bss`). Contrassegnata come Leggibile/Scrivibile.
`.rdata`	Dati in sola lettura (stringhe costanti, tabelle di importazione/esportazione).
`.rsrc`	Risorse dell'applicazione (icone, stringhe della GUI, sottomenu, o payload nascosti dal malware). Ispezionabile con *Resource Hacker*.

5. IMPORTAZIONI, ESPORTAZIONI E LIBRERIE DLL

Tipologie di Linking delle Librerie

- **Static Linking**: Il codice della libreria viene interamente copiato dentro l'eseguibile in fase di compilazione (file grande ma autonomo). Common in Linux.
- **Dynamic Linking**: L'eseguibile specifica nel PE Header quali DLL e funzioni caricare all'avvio. Il sistema operativo carica le DLL necessarie. Ispezionabile con **Dependency Walker**.
- **Runtime Linking**: Il programma contatta le DLL solo durante l'esecuzione usando le API `LoadLibraryA` e `GetProcAddress`. **Molto usata dai malware e dagli unpacker** per nascondere le funzioni realmente impiegate.

DLL Windows Comuni e Rilevanti

DLL	Funzionalità e Rilevanza per l'Analisi
`Kernel32.dll`	Funzioni core del sistema (gestione memoria, file, processi, hardware).
`Advapi32.dll`	Accesso ai componenti avanzati di Windows: Registro di sistema e Service Manager.
`User32.dll`	Componenti dell'interfaccia utente. Contiene API come `SetWindowsHookEx` (usata dai **Keylogger** per intercettare i tasti).
`Ntdll.dll`	Interfaccia diretta con il Kernel di Windows. Se importata direttamente indica intenzioni di manipolazione processi/evasione.
`Ws2_32.dll` / `Wininet.dll`	Librerie di rete (socket TCP/IP la prima, protocolli HTTP/FTP la seconda). Rilevano attività C2 o esfiltrazione.